



早稲田大学
WASEDA University

WASEDA University

Waseda Research Institute for Science and Engineering

Fundamentals of Programming

孫 鶴鳴

SUN Heming

Apr. 17th



Outline

- ◆ Variables
- ◆ Input & Output
- ◆ Operators

Variables

Variables

- Variables are used to declare values and strings
- Declaration is necessary before using variables.
- Type of Variables
- Type can be divided by :

Type	Details
int	Integer
long	Long integer
float	Floating point numbers
double	Double precision
char	Character

How many bytes
for each type?



- Variables Declaration :

Type Name;

For Example
int height, weight;
double height_m;

Hello.c

```
#include <stdio.h>
```

All C inbuilt functions which are declared in stdio.h header file can be used

```
int main(void)
```

your program begins executing at the beginning of main

```
{  
    printf("hello, world¥n");
```

¥ could be \ (back slash)
according the computer
environment

```
    return 0;  
}
```

⇒ Click "Save" icon to save the file

Main body

On most operating systems returning 0 is a success status like saying "The program worked fine".

Exercise1

```
#include<stdio.h>
```

```
int main(void)  
{
```

```
    int height;  
    int weight;
```

To store the data of height
Declare “Height” by Int Type

```
    height = 175;  
    weight = 65;
```

Assign the value to “Height”

```
    printf("Height is %d\n", height);
```

```
    return 0;
```

```
}
```

Try to print “Height is 175cm,
weight is 65kg”

Variable Naming Rules

- ◆ Names are made up of letters and digits; the first character must be a letter. The underscore ``_'' counts as a letter
- ◆ Upper and lower case letters are distinct, so x and X are two different names
 - ❖ Traditional C practice is to use lower case for variable names, and all upper case for **symbolic constants**.
- ◆ Cannot use **reserved words** as variables names
 - ❖ Keywords which have their predefined meaning

Reserved Words in C

C KEYWORDS OR RESERVED WORDS

auto break case char
const continue default do
int long register return
short signed sizeof static
struct switch typedef union
unsigned void volatile while
double else enum extern
float for goto if

Outline

- ◆ Variables
- ◆ Input & Output
- ◆ Operators

Output

Output

`printf( )` : To output variable values and characters

characters

```
printf("hello, world¥n");
```

- contents within "...." will be printed
- change line 「¥n」.

variable values

```
printf("%d¥n", height);
```

- Output variable 「height」' value.
- %: **specifier**
- define the type
and the interpretation of the value

specifier	detials
%d	decimal integer
%x	hexadecimal integer
%f	Decimal floating point
%e	exponent
%c	character

Exercise2

```
#include<stdio.h>

int main(void)
{

    int height = 175;

    printf("Height is %d\n", height);
    printf("Height is %x\n", height);

    double height_d = 175.0;

    printf("Height is %f\n", height_d);
    printf("Height is %e\n", height_d);

    return 0;

}
```

Input From the Keyboard

Input from the keyboard

`scanf( )` : input the value of variable from the keyboard

```
scanf("%d", &height);
```

- The value inputted by the keyboard will be assigned to `height`
- `&` before the variables

Specifies	details
<code>%d</code>	Decimal integer
<code>%f</code>	Decimal floating point
<code>%lf</code>	Decimal double floating point
<code>%c</code>	character

Scanf

```
...  
int height;  
int weight;  
printf("Input height (cm) >");  
scanf("%d", &height);  
...
```

The value inputted by the keyboard
will be assigned to 'height'

⇒ 「Input height (cm) >」 will be shown in the screen
Input the value by the keyboard
Push , then the value will be assigned to 「height」

Exercise3

bmi1.c

```
#include <stdio.h>
int main (void)
{
    int height;
    int weight;
    printf("Input height (cm) >");
    scanf("%d", &height);
    printf("Input weight (kg) >");
    scanf("%d", &weight);
    printf("height : %d cm¥n", height);
    printf("weight : %d kg¥n", weight);
    return 0;
}
```

Exercise4

```
#include <stdio.h>

int main()
{
    int a, b, c;
    printf("Enter the first value:");
    scanf("%d", &a);
    printf("Enter the second value:");
    scanf("%d", &b);
    c = a + b;
    printf("%d + %d = %d\n", a, b, c);
    return 0;
}
```

Compile

After compiling, the executive file 「a.out」 will be created

```
$ gcc bmi1.c ↵  
$ ./a.out ↵ ← the executive file of bmi1.c
```

⇒ However, if we compile a second source file,
(a.out) will be re-written

```
$ gcc bmi2.c ↵  
$ ./a.out ↵ ← the executive file of bmi2.c
```

By using 「-o」 option, the executive file can be named

```
gcc -o (executive file) (source file)
```

```
$ gcc -o bmi1 bmi1.c ↵  
$ ls ↵  
... bmi1 bmi1.c ...  
$ ./bmi1 ↵
```

Outline

- ◆ Variables
- ◆ Input & Output
- ◆ Operators

Arithmetic Operators

◆ $A = 10, B = 20$

Operator	Description	Example
+	Adds two operands.	$A + B = 30$
-	Subtracts second operand from the first.	$A - B = -10$
*	Multiplies both operands.	$A * B = 200$
/	Divides numerator by de-numerator.	$B / A = 2$
%	Modulus Operator and remainder of after an integer division.	$B \% A = 0$
++	Increment operator increases the integer value by one.	$A++ = 11$
--	Decrement operator decreases the integer value by one.	$A-- = 9$

Precedence of Arithmetic Operators

Operator(s)	Operation(s)	Order of evaluation (precedence)
()	Parentheses	Evaluated first. If the parentheses are nested, the expression in the <i>innermost</i> pair is evaluated first. If there are several pairs of parentheses “on the same level” (i.e., not nested), they’re evaluated left to right.
*/%	MultiplicationDivision Remainder	Evaluated second. If there are several, they’re evaluated left to right.
+-	AdditionSubtraction	Evaluated third. If there are several, they’re evaluated left to right.
=	Assignment	Evaluated last.

Bitwise Operators

◆ $A = 60$ (00111100), $B = 13$ (00001101)

Operator	Description	Example
&	Binary AND Operator copies a bit to the result if it exists in both operands.	$(A \& B) = 12$, i.e., 0000 1100
	Binary OR Operator copies a bit if it exists in either operand.	$(A B) = 61$, i.e., 0011 1101
^	Binary XOR Operator copies the bit if it is set in one operand but not both.	$(A \wedge B) = 49$, i.e., 0011 0001
~	Binary One's Complement Operator is unary and has the effect of 'flipping' bits.	$(\sim A) = \sim(60)$, i.e., -0111101
<<	Binary Left Shift Operator. The left operands value is moved left by the number of bits specified by the right operand.	$A << 2 = 240$ i.e., 1111 0000
>>	Binary Right Shift Operator. The left operands value is moved right by the number of bits specified by the right operand.	$A >> 2 = 15$ i.e., 0000 1111

Logical Operators

Operator	Description
&&	Called Logical AND operator. If both the operands are non-zero, then the condition becomes true.
	Called Logical OR Operator. If any of the two operands is non-zero, then the condition becomes true.
!	Called Logical NOT Operator. It is used to reverse the logical state of its operand. If a condition is true, then Logical NOT operator will make it false.

Exercise5

if x is 1 and y is 2,

then x & y is ?

x && y is ?

Assignment Operators

Operator	Description	Example
=	Simple assignment operator. Assigns values from right side operands to left side operand	$C = A + B$ will assign the value of $A + B$ to C
+=	Add AND assignment operator. It adds the right operand to the left operand and assign the result to the left operand.	$C += A$ is equivalent to $C = C + A$
-=	Subtract AND assignment operator. It subtracts the right operand from the left operand and assigns the result to the left operand.	$C -= A$ is equivalent to $C = C - A$
*=	Multiply AND assignment operator. It multiplies the right operand with the left operand and assigns the result to the left operand.	$C *= A$ is equivalent to $C = C * A$
/=	Divide AND assignment operator. It divides the left operand with the right operand and assigns the result to the left operand.	$C /= A$ is equivalent to $C = C / A$

Assignment Operators (cont.)

<code>%=</code>	Modulus AND assignment operator. It takes modulus using two operands and assigns the result to the left operand.	<code>C %= A</code> is equivalent to <code>C = C % A</code>
<code><<=</code>	Left shift AND assignment operator.	<code>C <<= 2</code> is same as <code>C = C << 2</code>
<code>>>=</code>	Right shift AND assignment operator.	<code>C >>= 2</code> is same as <code>C = C >> 2</code>
<code>&=</code>	Bitwise AND assignment operator.	<code>C &= 2</code> is same as <code>C = C & 2</code>
<code>^=</code>	Bitwise exclusive OR and assignment operator.	<code>C ^= 2</code> is same as <code>C = C ^ 2</code>
<code> =</code>	Bitwise inclusive OR and assignment operator.	<code>C = 2</code> is same as <code>C = C 2</code>

Misc. Operators

Operator	Description	Example
sizeof()	Returns the size of a variable.	sizeof(a), where a is integer, will return 4.
&	Returns the address of a variable.	&a; returns the actual address of the variable.
*	Pointer to a variable.	*a;
? :	Conditional Expression.	If Condition is true ? then value X : otherwise value Y

x? y : z

y if x is nonzero,
z otherwise